



ITES
RENÉ DESCARTES

CARRERA:
LICENCIATURA EN PROGRAMACIÓN
Y WEB MASTER

DOCENTE:
Mario Alonso Segovia Gutiérrez

MATERIA:
Proyecto de una Plataforma Virtual

ALUMNO:
CASTILLO PINZON CARLOS ALBERTO

FECHA:
12-06-2026

Introducción

A medida que las aplicaciones de software crecen en escala, funcionalidad y número de usuarios, la organización del código fuente se vuelve un factor crítico para el éxito del proyecto. Desarrollar sin una estructura clara produce sistemas rígidos, propensos a errores y difíciles de mantener, un fenómeno conocido comúnmente como "código espagueti". Para mitigar estos riesgos, la ingeniería de software recurre a los patrones de arquitectura.

Un patrón de arquitectura representa una solución abstracta y probada a problemas recurrentes de organización estructural en sistemas de software. Estos patrones actúan como planos de construcción que definen de manera anticipada cómo se estructurarán los componentes, cuáles serán sus responsabilidades principales y de qué forma interactuarán y se comunicarán entre sí. La adopción estratégica de un patrón arquitectónico impacta de forma directa en atributos de calidad esenciales como la escalabilidad, la modularidad, la facilidad de mantenimiento y el rendimiento global, garantizando que el sistema pueda evolucionar de manera sostenible a lo largo de su ciclo de vida.

PARTE 1 — INVESTIGACIÓN

MVC (Model-View-Controller)

Definición:

Es un patrón que divide la aplicación en tres componentes independientes para separar la lógica de negocio de la interfaz de usuario.

Estructura general:

Modelo → Controlador → Vista.

Componentes principales:

- Modelo: gestiona datos y reglas del negocio.
- Vista: presenta la información al usuario.
- Controlador: recibe solicitudes y coordina la comunicación.

Forma de funcionamiento:

El usuario interactúa con la vista; el controlador procesa la petición y actualiza el modelo; posteriormente la vista muestra los resultados.

Ventajas:

- Separación clara de responsabilidades.
- Facilita el mantenimiento.
- Permite reutilizar componentes.
- Favorece el trabajo en equipo.

Desventajas:

- Puede resultar complejo en aplicaciones pequeñas.
- Requiere una correcta organización.

Casos de uso:

- Aplicaciones web.
- Sistemas desarrollados con Laravel, ASP.NET MVC o Spring MVC.

Arquitectura en Capas

Definición:

Organiza el sistema en niveles jerárquicos donde cada capa tiene funciones específicas.

Estructura general:

- Presentación.
- Lógica de negocio.
- Acceso a datos.
- Base de datos.

Componentes principales:

- Capa de presentación.
- Capa de negocio.
- Capa de persistencia.
- Capa de datos.

Forma de funcionamiento:

Cada capa interactúa únicamente con la inmediatamente inferior o superior.

Ventajas:

- Modularidad.
- Facilidad de mantenimiento.
- Mayor organización.
- Facilita pruebas.

Desventajas:

- Puede afectar el rendimiento.
- Incrementa la cantidad de código.

Casos de uso:

- Sistemas empresariales.
- Sistemas administrativos.
- Aplicaciones de escritorio.

Arquitectura Monolítica

Definición:

Consiste en desarrollar toda la aplicación como una única unidad desplegable.

Estructura general:

Todos los módulos forman parte del mismo proyecto.

Componentes principales:

- Interfaz.
- Lógica de negocio.
- Acceso a datos.

Forma de funcionamiento:

Todos los componentes se ejecutan dentro del mismo entorno.

Ventajas:

- Fácil implementación inicial.
- Menor complejidad.
- Despliegue sencillo.

Desventajas:

- Dificil escalabilidad.
- Mantenimiento complejo al crecer.
- Cambios afectan a todo el sistema.

Casos de uso:

- Proyectos pequeños.
- MVP (Producto Mínimo Viable).
- Equipos reducidos.

Microservicios

Definición:

Arquitectura que divide el sistema en servicios pequeños e independientes.

Estructura general:

Cada servicio posee su propia lógica y, en muchos casos, su propia base de datos.

Componentes principales:

- Servicios independientes.
- APIs.
- Balanceadores.
- Herramientas de monitoreo.

Forma de funcionamiento:

Los servicios se comunican mediante protocolos como HTTP o mensajería.

Ventajas:

- Alta escalabilidad.
- Independencia entre servicios.
- Flexibilidad tecnológica.
- Despliegues independientes.

Desventajas:

- Mayor complejidad.
- Costos de infraestructura elevados.
- Difícil monitoreo.

Casos de uso:

- Plataformas de streaming.
- Ecommerce de gran escala.
- Aplicaciones bancarias.

Cliente-Servidor

Definición:

Modelo donde uno o varios clientes solicitan servicios a un servidor central.

Estructura general:

Cliente ↔ Servidor.

Componentes principales:

- Cliente.
- Servidor.
- Red de comunicación.

Forma de funcionamiento:

El cliente realiza solicitudes y el servidor procesa y responde.

Ventajas:

- Administración centralizada.
- Seguridad controlada.
- Fácil actualización del servidor.

Desventajas:

- Dependencia del servidor.
- Posibles cuellos de botella.
- Punto único de falla.

Casos de uso:

- Aplicaciones web.
- Sistemas de correo electrónico.
- Bases de datos centralizadas.

PARTE 2 — CUADRO COMPARATIVO

Aspecto	MVC	Arquitectura en Capas	Monolítica	Microservicios	Cliente-Servidor
Objetivo principal	Separar responsabilidades	Organizar funciones por niveles	Integrar todo en una sola aplicación	Dividir el sistema en servicios independientes	Centralizar servicios
Estructura	Modelo, Vista y Controlador	Capas jerárquicas	Unidad única	Servicios autónomos	Cliente y servidor
Organización del código	Alta	Alta	Media	Muy alta	Media
Escalabilidad	Moderada	Moderada	Baja	Muy alta	Moderada
Mantenimiento	Fácil	Fácil	Difícil en sistemas grandes	Independiente por servicio	Moderado
Complejidad	Media	Media	Baja	Alta	Media
Facilidad de implementación	Alta	Moderada	Muy alta	Baja	Alta
Rendimiento	Bueno	Bueno	Alto	Variable	Dependiente del servidor
Reutilización	Alta	Alta	Baja	Muy alta	Moderada
Casos de uso recomendados	Aplicaciones web	Sistemas empresariales	Proyectos pequeños	Sistemas masivos	Aplicaciones distribuidas
Ventajas	Separación clara	Modularidad	Simplicidad	Escalabilidad	Administración centralizada
Desventajas	Organización compleja	Más capas	Difícil crecimiento	Complejidad elevada	Dependencia del servidor

PARTE 3 — ANÁLISIS

1. ¿Cuál de los patrones consideras más adecuado para una plataforma web desarrollada por un equipo pequeño y por qué?

Considero que la arquitectura MVC es la más adecuada, ya que facilita la organización del código mediante la separación entre datos, interfaz y lógica de control. Además, es compatible con numerosos frameworks modernos y permite que equipos pequeños trabajen de manera eficiente sin añadir complejidad innecesaria.

2. ¿Qué ventajas ofrecen los microservicios frente a una arquitectura monolítica?

Los microservicios ofrecen mayor escalabilidad, ya que cada servicio puede ampliarse de manera independiente. También permiten realizar despliegues sin afectar al sistema completo, facilitan el mantenimiento y brindan flexibilidad para utilizar diferentes tecnologías según las necesidades de cada servicio.

3. ¿Por qué MVC continúa siendo uno de los patrones más utilizados en aplicaciones web?

Porque proporciona una estructura clara y ordenada que facilita el mantenimiento y la reutilización del código. Asimismo, numerosos frameworks populares se basan en este patrón, lo que simplifica el desarrollo y reduce la curva de aprendizaje.

4. ¿Qué riesgos podrían existir al implementar una arquitectura demasiado compleja para un proyecto pequeño?

Una arquitectura excesivamente compleja puede incrementar los costos de desarrollo, dificultar el mantenimiento, requerir más recursos técnicos y retrasar la entrega del proyecto. Además, puede generar soluciones innecesarias para problemas que aún no existen.

5. ¿Crees que existe un patrón mejor que todos los demás? Justifica tu respuesta.

No existe un patrón universalmente mejor. La elección depende de factores como el tamaño del proyecto, el presupuesto, la cantidad de usuarios y la experiencia del equipo. Cada patrón posee fortalezas y limitaciones, por lo que la mejor opción será aquella que responda adecuadamente a las necesidades específicas del sistema.

PARTE 4 — CASO PRÁCTICO

Sistema seleccionado: Ecommerce

Para desarrollar un sistema de comercio electrónico utilizaría una arquitectura basada en microservicios.

Justificación:

- Número de usuarios: Un ecommerce puede recibir miles de usuarios simultáneamente, especialmente durante promociones o temporadas altas.
- Escalabilidad: Los módulos de catálogo, pagos, inventario y envíos pueden escalar de forma independiente.
- Complejidad: El sistema integra múltiples procesos que pueden desarrollarse como servicios separados.
- Mantenimiento: Permite actualizar componentes específicos sin afectar el funcionamiento general.
- Costos: Aunque la inversión inicial es mayor, resulta rentable a largo plazo debido a su flexibilidad.
- Tamaño del equipo: Facilita que varios equipos trabajen simultáneamente en diferentes servicios.

Conclusión

La realización de esta actividad permitió comprender que la arquitectura de software constituye una decisión estratégica dentro del desarrollo de sistemas. Los patrones arquitectónicos proporcionan soluciones probadas que ayudan a organizar mejor el código, distribuir responsabilidades y mejorar la calidad del producto final.

El patrón que me pareció más interesante fue el de microservicios, debido a su capacidad para adaptarse al crecimiento de aplicaciones complejas y a su flexibilidad para el mantenimiento. Sin embargo, también comprendí que no siempre representa la mejor alternativa, especialmente en proyectos pequeños.

La arquitectura influye directamente en aspectos como la escalabilidad, el rendimiento, la mantenibilidad y la facilidad para incorporar nuevas funcionalidades. Por ello, seleccionar adecuadamente una arquitectura desde las etapas iniciales del proyecto permite reducir riesgos, optimizar recursos y garantizar que el sistema pueda evolucionar conforme cambien las necesidades de los usuarios y de la organización.